



COLLEGE OF
COMPUTER STUDIES

Week 3

FOPR111 – Fundamentals of Programming

Introduction to C

Objectives:

By the end of this lesson, you should be able to:

- Familiarize with the history of C language
- Define Features of C language
- Know the uses of C language.
- Understand the C programming Development Cycle
- Identify the Types and Levels of Programming Language



C LANGUAGE

C Language

- The C Language is a general purpose, high level and structured oriented programming language developed at Bell laboratories in 1972, by Dennis M. Ritchie.
- C programming features, where derived from an earlier language called 'B' (BCPL-Basic Combined Programming Language).
- C language was invented for implementing UNIX operating system.
- In 1978, Dennis Ritchie and Brian Kernighan published the first edition “ The C Programming Language” and commonly known as “K & R C”
- In 1983, the ANSI (American National Standard Institute) established a committee to provide a modern, comprehensive definition of C. The resulting definition is the ANSI C, completed in 1988

Features of C

The following are the standard features of C Language as follows.

1. Portability: Programs written in C language can be run on any compiler with little or no modifications of a program.
2. Flexibility: C language has number of reserved keywords, which helps the programmers to take control over the usage of language and to modify the structure of program.
3. Modularity: C language provides feature of modularity, i.e. we can breakdown a large program into manageable modules. This feature provides easy to understand and debugging of the program will become faster.

Features of C

The following are the standard features of C Language as follows.

4. Robustness: C language is a robust language, with rich set of built-in functions and operators that can be used to write any complex programs.
5. Extensibility: In C language program, the new features can be added at any time by the programmer to the existing program.
6. Efficiency and effectiveness: Programs written in C language are fast and efficient, due to the variety of data types and powerful operators.

Application of C

C is a general-purpose, high-level language that was originally developed by Dennis M. Ritchie to develop the UNIX operating system at Bell Labs. C was originally first implemented on the DEC PDP-11 computer in 1972.

Some of applications of the C language are:

1. C language is used for creating computer applications.
2. C language is used to write embedded software's.
3. It is also used in developing verification software, test code, simulators etc. for various applications and hardware products.
4. Used for creating Language Compilers, Assemblers, Text editors, Network Drivers, Language interpreters, Database programs, and utilities.
5. C is used to implement different Operating System Operations.
6. UNIX Kernel is completely developed in C Language



C Programming Development Cycle

C Programming Development Cycle

Executing a program written in C Language involves a series of steps. They are:

1. Creating the program
2. Compiling the program
3. Linking your program with whatever function needed from the library
4. Executing the program

C Programming Development Cycle

1. Creating the program

The software used to write a program is known as **text editor**. A text editor helps us to enter, change and store character data. After we complete the program, we save our file to disk. This file will be input to the compiler and this file is known as source file.

C Programming Development Cycle

2. Compiling the program

The compiler software (Program) translates the source file into its equivalent machine language code and also it displays if any syntactical errors in the program.

C Programming Development Cycle

3. Linking the program:

Linking is process of putting together other program files and functions that are required by the source program. For example, if the program is using **printf()** function, then the object code of this function should be brought from the **stdio.h** library of the system and linked it to the source program.

C Programming Development Cycle

4. Executing the program

Once the .EXE file is created and stored on the disk, it is ready for execution. When we execute it, it is first brought from the disk into the memory (RAM) by an operating system component called Program Loader then asks for input parameters from the user, processes it and produces the results to user.



TYPES OF PROGRAMMING LANGUAGES

TYPES OF PROGRAMMING LANGUAGES

1. Procedural Languages:

Organize code into procedures or functions.

Examples: C, Pascal, Fortran.



2. Object-Oriented Languages (OOP):

Focus on objects, promoting modularity and reusability.

Examples: Java, Python, C++.



TYPES OF PROGRAMMING LANGUAGES

3. Functional Languages:

Emphasize mathematical functions and immutability.

Examples: Haskell, Lisp, Scala.



4. Scripting Languages:

Prioritize ease of use and rapid development.

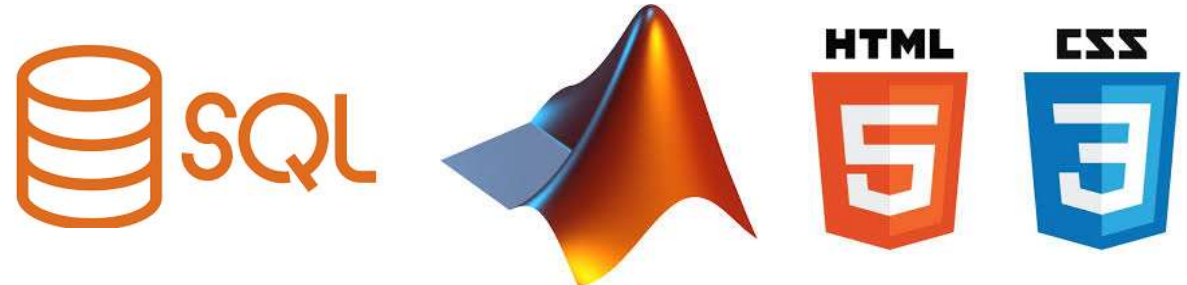
Examples: Python, Ruby



5. Domain-Specific Languages:

Tailored for specific Problem domains

Examples: SQL, MATLAB, HTML/CSS



Level of Programming Language

A. Middle Level

Middle level languages do not provide all the built-in functions found in high level languages, but to provide all building blocks needed to produce the result desired.

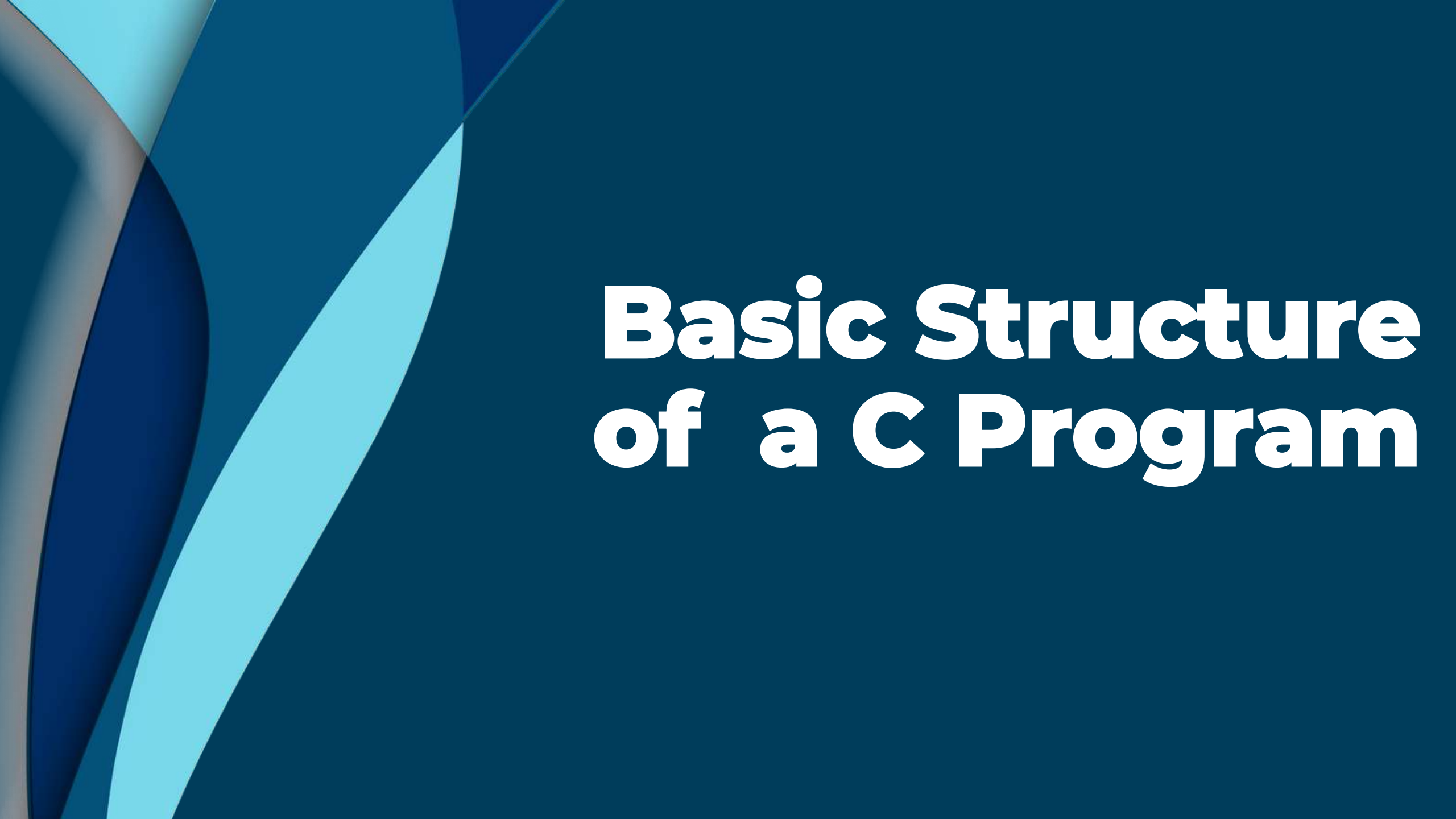
B. High Level

High level languages provide almost everything that the programmer might need to do as already built into the language.

C. Low Level

Low level languages provide nothing other than access to the machines basic instruction set

.



Basic Structure of a C Program

Basic Structure of a C Program

```
#include <stdio.h>
```

`#include <stdio.h>` is a **header file library** that lets us work with input and output functions, such as `printf()` (used in line 4). Header files add functionality to C programs.

```
int main() {
```

`main()`. This is called a **function**. Any code inside its curly brackets `{}` will be executed.

```
// your code  
printf("Hello World!");
```

`printf()` is a **function** used to output/print text to the screen

```
return 0;  
}
```

`return 0` ends the `main()` function.

Note that: Every C statement ends with a semicolon ;

Header Files Inclusion

All lines that start with # are processed by a preprocessor which is a program invoked by the compiler. The example, the preprocessor copies the preprocessed code of `stdio.h` to our file. The .h files are called header files in C.

Some of the C Header files:

`stddef.h` - Defines several useful types and macros.

`stdint.h` - Defines exact width integer types.

`stdio.h` - Defines core input and output functions

`stdlib.h` - Defines numeric conversion functions, pseudo-random number generator, and memory allocation

`string.h` - Defines string handling functions

`math.h` - Defines common mathematical functions.



Difference Between C and C++

Difference Between C and C++

C++ was created to add the OOPs concept into the C language so they both have very similar syntax with a few differences. The following are some of the main differences between C and C++ Programming languages.

Feature	C	C++
Paradigm	Procedural programming	Procedural, Object-Oriented, Generic
OOPs	Not Supported	Supports OOPs concepts with classes, inheritance, polymorphism, and encapsulation.
Memory Management	Manual using malloc(), calloc(), free() etc.	Both Manual and Automatic (using new, delete for dynamic allocation and deallocation)
Function Overloading	Not Supported (function names must be unique.)	Supports function overloading.
Operator Overloading	Not Supported (every operator performs unique operation.)	Supports operator overloading, allows custom behaviors for operators like +, -, *, etc.
Access Control	Doesn't have any access control mechanism.	Supports access control using keywords like private, public , protected.



Thank you

Let's do something great